

Computer Science Bibles

2021/07/10 ~

박지원

목차

<6강_클래스 다루기>

1. 인터페이스

1) 정의

= 설계도

- Script = Class를 확장시켜 Script = (Class OR Interface)로 정의할 수 있습니다.
- Interface를 구현하는 Script의 이름 앞에는 'I'를 붙이기로 약속했습니다.
- method 이름만 구현하고 , 아무 기능을 구현하지 않기 때문에 반드시 다른 Script(=Class)에 상속 받아서 사용해야 합니다.
- 어떤 Script(=Class)가 Interface를 상속 받았다면 해당 Script는 Interface에 존재하는 모든 Method를 구현해야 합니다. 이 과정이 Override이지만 C#에서는 Override를 붙이지 않아도 됩니다.

```
public class GoldItem : MonoBehaviour, IItem {
```

- GoldItem(=Class)은 MonoBehaviour(=Class)와 IItem(=Interface)을 상속받습니다.
- GoldItem은 IItem에 존재하는 모든 Method를 구현해야 합니다.

2) 인터페이스를 사용해야 하는 이유 (게임을 예시로 설명)

- 이 게임에는 아이템이 1,000 종류가 있고 플레이어와 닿으면 해당 아이템이 자동으로 사용됩니다.
- 아이템이 자동으로 사용되는 기능은 Use 라는 Method로 구현합니다.

① 기존의 허접한 방식 : If문 내부에 1,000개의 분기 생성

= Player Script에 OnTriggerEnter()함수를 만들고 , 그 안에 1,000개의 if문을 이용하여 조건에 맞는 아이템을 작동시킬 것 입니다. 1,000개의 아이템마다의 Script에는 당연히 Use 함수를 구현해야 합니다.

② Interface를 사용하는 훌륭한 방식 : If문 내부에 단 1개의 분기 생성

= Interface의 다형성과 Interface를 상속 받은 Class는 반드시 Interface의 Method를 구현해야 하는 방식으로 인해 분기 없이 코드를 작성할 수 있습니다. 1,000개의 아이템마다의 Script에는 당연히 Use 함수를 구현해야 합니다.

<허접한 기존의 방식>

```
void OnTriggerEnter2D(Collider2D other)
{
    GoldItem goldItem = other.GetComponent<GoldItem>();

    if(goldItem != null)
    {
        goldItem.Use();
    }

    HealthKitItem healthKitItem = other.GetComponent<HealthKitItem>();

    if(healthKitItem != null)
    {
        healthKitItem.Use();
    }
}
```

<Interface를 사용한 훌륭한 방식>

```
void OnTriggerEnter2D(Collider2D other)
{
    IItem item = other.GetComponent<IItem>();

    if(item != null)
    {
        item.Use();
    }
}
```

- Reference : <https://www.udemy.com/course/retr0-unity/learn/lecture/8609448#overview>

<10강_변수 사용 시 고려할 사항>

1. Implicit Declaration (=묵시적 선언 , =암시적 선언)을 사용하지 않습니다.

묵시적 선언(implicit declaration)

”

- 묵시적 선언(implicit declaration): 선언문을 사용하는 것이 아니라 디폴트 규칙에 의해서 변수에 속성을 부여함

- ex) FORTRAN에서는 변수 이름을 그냥 사용하며

변수 이름이 I, J, K, L, M, N으로 시작하면 정수 타입으로 간주하는 등 디폴트 규칙이 있음,

- 저는 원래 implicit declaration을 사용하지 않고 있었습니다.

명시적 선언(explicit declaration)

”

- 명시적 선언(explicit declaration): 선언문을 사용하여 변수에 속성을 부여함

- ex) int min(min, min2);, this.c = c;(this를 명시하여 표기)

2. 전역변수와 지역변수 초기화

1) 전역변수

= int는 0 , char는 NULL(=0), string은 빈 문자열로 초기화 되며 , 배열은 이에 연속입니다.

- 컴파일러가 알아서 초기화 해줍니다.

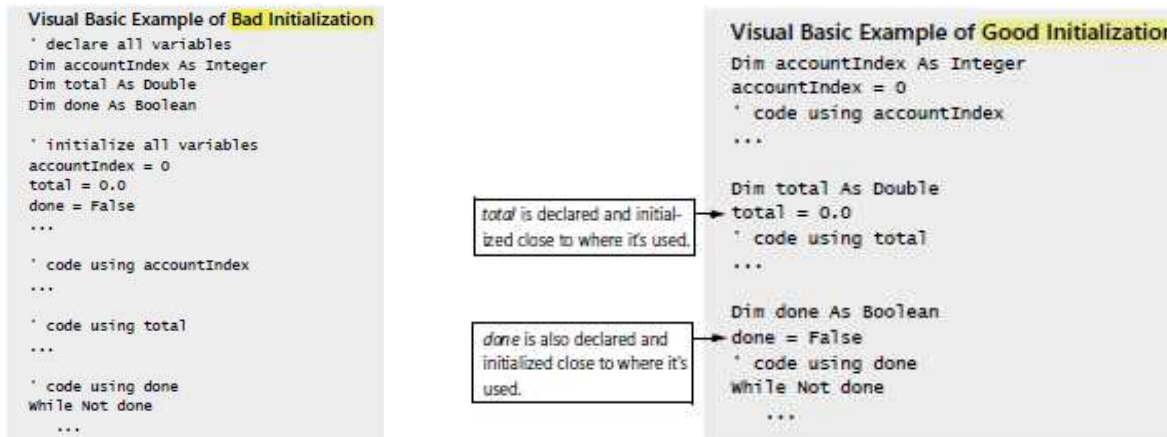
2) 지역변수

= 초기화를 하지 않으면 Warning이 발생하거나 ERROR가 발생하므로 반드시 초기화를 하고 사용합시다.

3. 변수 초기화 가이드라인

1) 변수는 선언 할 때 초기화 합니다.

2) 변수가 처음 사용되는 곳 근처에서 초기화 합니다.



3) 가능하면 const를 많이 사용합니다.

= 변수가 초기화 된 이후 다른 값으로 할당되는 것을 막을 수 있습니다.

- 이제 해당 변수는 Constant가 되므로 값을 변경시킬 수 없습니다.

4) 이름상수(ex : YEAR 2021)는 프로그램의 시작 부분에서 초기화 합니다.

4. 변수의 수명(범위)

= 변수의 수명은 가능한 한 짧게 유지해야 합니다.

- 코드의 이해가 쉬워지며 , 초기화 오류가 발생할 가능성을 줄여줍니다.

- 변수의 값을 변경하려는 위치 사이에서 변수가 부주의하게 변경될 가능성을 줄일 수 있습니다.

5. 내가 경험한 이름의 진가

- 오랜만에 내가 작업한 코드를 다시 작업할 때 , 함수명과 변수명을 잘 써놓으면 진짜 바로 이해가 됩니다.
- 함수명만 보고 내부구조를 이해하지 않고 , 해당 함수를 다른 곳에서 의심하지 않고 사용하고 실행했을 때 이상 없이 잘 돌아가는 걸 경험했습니다. 미래의 나를 위해서 완벽하게 함수를 작성해야 합니다.

<11강_변수 이름의 기능>

타입	규칙	예시
변수 & 객체	소문자 & '_'	<code>int spawn_monster_min</code>
함수(메소드) & 클래스 & 프로퍼티	구 마다 대문자	<code>void SpawnMonster</code>
Unity GameObject & Tag		<code>NightWalker</code>
상수	모두 대문자 & '_'	<code>const int ENGLISH_SCORE</code>
Bool_Type	앞에 is or has를 붙입니다.	<code>bool is_true(변수)</code> <code>bool IsDead(메소드)</code>
자료구조	소문자 & '_' & 복수형 ▼	<code>List<int> lists</code>

- 자료구조를 굳이 복수형으로 만들어야 할까?

1. 좋은 이름(변수, 함수)을 짓는 방법

= 누구나 객체의 이름을 보고 그것이 어떤 역할을 하는지 빠르고 명확하게 이해할 수 있도록 작성합니다.

- 구체적인 단어를 사용해야 하며, 모호한 단어는 피해야 합니다.

- 단위를 포함하는 경우에는 단위를 이름에 포함시킵니다. ex) start_ms

- 연구 결과 이상적인 이름의 길이가 있지만, **길이에 신경 쓰지 말고 작성**합니다. 작가도 올림픽에서 각 나라의 대표 팀에서 획득한 최고 점수를 표현하는 변수의 이름은 maximum_number_of_points_in_modern_olympics로 표현했습니다.

1) 좋은 함수 이름을 짓는 방법

- **함수가 하는 모든 것을 표현**합니다.

- **동사형**으로 작성하되, 애매하거나 모호한 표현을 갖는 동사를 사용하지 맙시다.

2) 좋은 변수 이름을 짓는 방법

- Variable Space가 작은 지역변수는 임시 이름(ex : tmp)을 상관해도 괜찮습니다.

2. 자주 쓰는 변수에 대한 고찰

1) 반복문 i, j, k

- 간단한 반복문에서는 i, j, k가 괜찮고 오랜 관습이지만, 복잡한 반복문인 경우는 좀 더 의미 있는 변수를 사용하길 권장합니다.

- i, j, k끼리의 index 혼선(ex : i인데 j를 쓰는 실수)을 겪은 적이 많은데 의미 있는 변수 이름을 사용하면 이 문제를 회피할 수 있습니다.

2) 임시 변수

- 개발자들은 보통 임시 변수를 tmp로 짓고 별생각 없이 다루기 때문에 잦은 오류가 발생합니다.

- 임시 변수도 tmp 보다는 의미 있는 변수 이름으로 지어 이를 방지합니다.

3) 축약어

- 작가는 현대 프로그래밍에서 축약어는 필요하지 않다고 주장하면서도, 일관성 있고 규칙이 있는 축약어는 허용한다고 말하고 있습니다.

- 단어에서 한 문자를 없애는 방법으로는 축약하지 않습니다.

<12강_기본 데이터 형식>

1. 이름 상수 (=Named Constant)

ex) `const int APPLE_PRICE = 3100`

- 이름 상수를 지원하는 프로그래밍 언어에서는 반드시 일반 숫자(=magic number), 일반 문자(=magic string) 대신 이름 상수를 사용합니다.
- 이름 상수는 코드에서 변경이 쉬우며 , 코드의 가독성이 높아집니다.
- 0과 1은 일반 숫자를 사용합니다.
- 다른 type과 비교하지 않습니다.
- 데이터 선언에 이름 상수를 사용해야 합니다. 배열의 크기를 이름 상수로 선언했을 때 , 변경하기 용이하여 유지보수에 훌륭합니다.
- 명백한 Literal이여도 Literal은 절대 사용하지 말고 Named Constant를 사용합니다. (작가는 2,3,4 같은 숫자가 코드에 존재하는 것을 혐오합니다.)

2. 정수 Overflow

- 개발자는 서로 다른 정수형의 최댓값을 인식하고 있어야 합니다.
- 계산 중간 과정에서도 Overflow가 발생할 수 있으니 주의해야 합니다. (알고리즘 공부 과정에서 많이 경험)

Integer Type	Range
Signed 8-bit	-128 through 127
Unsigned 8-bit	0 through 255
Signed 16-bit	-32,768 through 32,767
Unsigned 16-bit	0 through 65,535
Signed 32-bit	-2,147,483,648 through 2,147,483,647
Unsigned 32-bit	0 through 4,294,967,295
Signed 64-bit	-9,223,372,036,854,775,808 through 9,223,372,036,854,775,807
Unsigned 64-bit	0 through 18,446,744,073,709,551,615

Java Example of Overflow of Intermediate Results

```
int termA = 1000000;  
int termB = 1000000;  
int product = termA * termB / 1000000;  
System.out.println( "( " + termA + " * " + termB + " ) / 1000000 = " + product );
```

3. C 방식 문자열

- C++ STL의 string은 C 방식 문자열의 단점을 대부분 제거했습니다. 그러므로 C++ STL의 string만을 사용합시다.
- C++ STL의 string을 사용한다면 C에서 문자열로 고생했던 대부분의 문제들이 발생하지 않습니다.
- C 방식 문자열은 문자를 모두 저장하고 마지막에 '\0'(=아스키코드의 0x00을 의미)을 1byte를 사용하여 저장합니다. 그러므로 "abcde"를 C 방식 문자열로 저장하면 크기가 6이고, C++ STL의 string으로 저장하면 크기가 5입니다.

4. 열거형 (Enum)

= Enumerated Types

- 열거형 내부의 각 멤버를 문자로 표현하면 컴파일러는 이들을 정수형 상수로 인식합니다.
- 관련 있는 상수들을 한 곳에 모아놓기 때문에 가독성이 향상 됩니다.

1) 일반 enum : 상수를 사용할 때 많이 사용합시다!!!

① Default

```
enum 요일
{
    월, 화, 수, 목, 금, 토, 일
};
```

- 값을 지정하지 않으면 가장 앞의 멤버가 0이 됩니다.
- 값을 지정하지 않으면 직전 멤버의 값에 + 1 한 값을 부여합니다.

② 사용자 값 지정

```
enum 요일
{
    월=1, 화, 수, 목, 금, 토, 일=10
};
```

- 값을 지정하지 않으면 가장 앞의 멤버가 0이 됩니다.
- 값을 지정하지 않으면 직전 멤버의 값에 + 1 한 값을 부여합니다.
- 값을 지정하면 해당 값을 상수로 갖습니다.

- 좋은 예시지만 실제 코드에서는 당연히 영어로 사용합니다.

<문제점>

```
enum ECar{
    BMW,
    AUDI,
    KIA,
    BENZ,
    HYUNDAE,
    PORSCHE,
};

enum EKoreaCar{
    KIA,
    HYUNDAE,
};

int main(void)
{
    int num = KIA; //error
    cout << "KIA : " << num << endl;
    return 0;
}
```

- 각각의 enum에 중복된 멤버인 KIA가 존재하여 ERROR가 발생합니다.

2) Class enum

- 기존 enum의 문제점(중복)을 해결하기 위해 등장했습니다.

```
enum class EColor
{
    RED,
    BLUE,
    GREEN,
    YELLOW,
}

enum class ERainbowColor
{
    RED,
    ORANGE,
}
```

이런식으로 enum이 선언, 정의 되어있다고 할때
EColor 열거형의 RED, ERainbowColor 열거형의 RED를 부를때 각각 열거형(enum)의 이름을 앞에 두고 부르면 됩니다. 간단하죠?

```
EColor::RED
ERainbowColor::RED
```

5. 정수와 소수

1) 분수와 소수

```
cout << 3 / 2 << "\n"; // 1
cout << 3.0 / 2.0 << "\n"; // 1.5
cout << 1.5 << "\n"; // 1.5

// *** 나쁜 코드 ***
cout << 3.0 / 2 << "\n"; // 1.5
```

<중요한 개념>

- 기본적으로 **정수**는 **Int** 취급 , **소수**는 **Double** 취급을 합니다.
- **정수로 이루어진 분수**는 **Int**로 취급하므로 매우 위험합니다.
- **소수로 이루어진 분수**는 **Double**로 취급하므로 안전합니다.
- 정수와 소수가 혼합되어 있는 분수는 다른 type 연산이므로 사용하지 맙시다.

2) 형 변환

```
cout << (double)(3 / 2) << "\n";
cout << (double)3 / (double)2;
```

1
1.5

- 3/2 연산을 먼저하고 double로 강제 형 변환(=cast)을 시키므로 (double) 1 과 같습니다.
- 아래의 코드처럼 작성해야 올바른 결과가 나오며 , 보통은 소수를 이용합니다.

3) 소수의 오류 : 1.15를 두 짝 자리에서 반올림 했지만 1.1인 오류

```
double a = 1.14;
double b = 1.15;
double c = 1.1500001;
double d = 1.1500000000000001;

printf("%.15f\n", d);
printf("%.1f %.1f %.1f %.1f", a , b , c , d);
```

1.1500000000000001
1.1 1.1 1.2 1.2

<13강_특이한 데이터 형식>

1. Struct (구조체)

1) 구조체를 사용하는 이유

- 서로 관련된 데이터를 한 곳으로 모으기 때문에 관리하기 용이합니다. 그 덕에 , 유지보수가 용이합니다.
- 매개변수 목록을 단순화할 수 있습니다.

Visual Basic Example of a Clumsy Routine Call Without a Structure

```
HardWayRoutine( name, address, phone, ssn, gender, salary )
```

And this is an example of the easy way to call a routine by using a structured variable that contains the elements of the first parameter list:

Visual Basic Example of an Elegant Routine Call with a Structure

```
EasyWayRoutine( employee )
```

2) 특이하게 , C#의 구조체는 클래스와 많이 비슷해졌지만 아직은 다릅니다.

① C++ Struct

```
struct str
{
    int a;    str obj;
    int b;    obj = { 1,2 };
};           cout << obj.a << " " << obj.b;
```

<- C++ 코드

- C++에서는 구조체를 사용하는 것과 클래스를 사용하는 것의 차이가 분명히 존재합니다.
- 클래스는 조금 복잡하지만 구조체는 데이터를 모아둔 집합체에 불과하여 매우 간단하고 제약도 적었습니다.

② C# Struct

```
public struct str
{
    public int a;
    public int b;
};           str obj;
           obj = { 1,2};
```

<- C# 코드

- 하지만 C#은 C++처럼 구조체의 사용이 자유롭지 않습니다. 값을 할당하려면 public을 구조체와 모든 멤버에 적용해야 하며 , 초기화도 클래스와 비슷한 형식으로 해야 합니다.
- 다시 말해 , C#의 구조체는 간단하지 않고 클래스와 비슷하게 불편합니다.

③ C# Class

```
public struct records
{
    public string p1_name;
    public string p1_score; //int형 데이터도 string으로 이용
    public string p2_name;
    public string p2_score;

    public records(string a , string b , string c , string d)
    {
        this.p1_name = a;
        this.p1_score = b;
        this.p2_name = c;
        this.p2_score = d;
    }
};
```

```
public class records
{
    public string p1_name;
    public string p1_score; //int형 데이터도 string으로 이용
    public string p2_name;
    public string p2_score;

    public records(string a , string b , string c , string d)
    {
        this.p1_name = a;
        this.p1_score = b;
        this.p2_name = c;
        this.p2_score = d;
    }
};
```

```
records tmp = new records(row[0] , row[1] , row[2] , row[3]);
```

- 좌측의 struct를 우측의 class로 바꾸어도 코드의 아무런 변화가 없습니다. 즉 , C#에서 struct와 class는 동일합니다.
- 그러므로 구조체로 만들 것을 클래스로 만들고 , 선언도 객체를 생성하며 하고 , 그 과정에서 생성자로 초기화합니다.

④ Reference

Classes Only:

- Can support inheritance
- Are reference (pointer) types
- The reference can be null
- Have memory overhead per new instance

Structs Only:

- Cannot support inheritance
- Are value types
- Are passed by value (like integers)
- Cannot have a null reference (unless Nullable is used)
- Do not have a memory overhead per new instance - unless 'boxed'

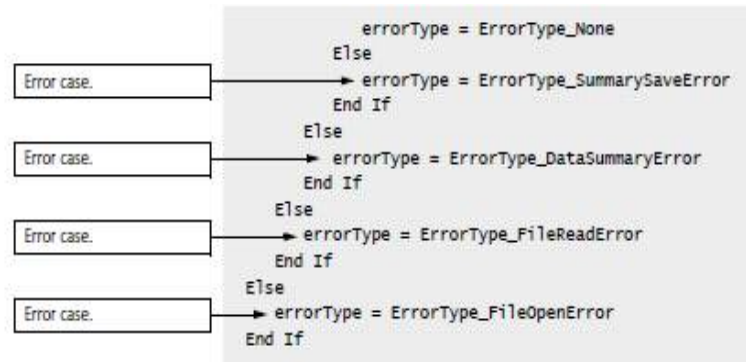
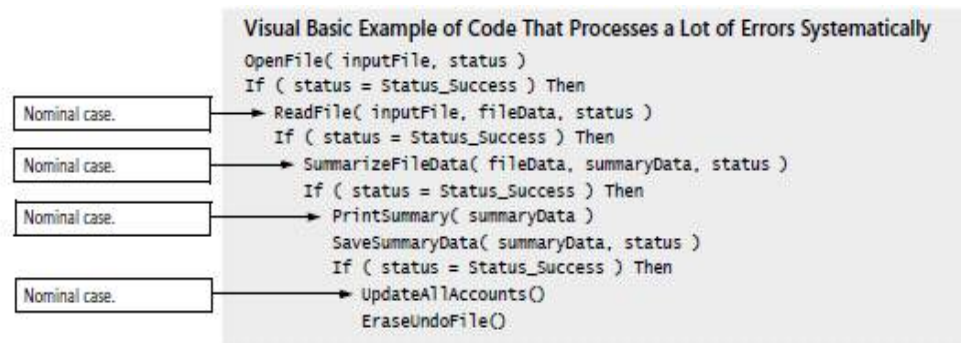
Both Classes and Structs:

- Are compound data types typically used to contain a few variables that have some logical relationship
- Can contain methods and events
- Can support interfaces

<15강~17강_Statement (명령문)>

1. 조건문 (If-Statement)

- 정상적으로 처리 되는 경우를 조건문의 앞쪽에 적습니다. 그렇게 되면 오류 조건이 조건문의 마지막에 쌓여있게 되는 좋은 현상이 나타납니다.
- 복잡한 조건은 Boolean 함수의 호출로 단순화 합니다.



C++ Example of an *if-then-else* Chain That Uses Boolean Function Calls

```
if ( IsControl( inputCharacter ) ) {
  characterType = CharacterType_ControlCharacter;
}
else if ( IsPunctuation( inputCharacter ) ) {
  characterType = CharacterType_Punctuation;
}
else if ( IsDigit( inputCharacter ) ) {
  characterType = CharacterType_Digit;
}
else if ( IsLetter( inputCharacter ) ) {
  characterType = CharacterType_Letter;
}
```

- case 다음에 오는 코드가 짧으면 case문의 구조를 명확하게 하는 데 도움을 줍니다. 그러므로 코드가 길어지면 함수를 만들어서 case에서 호출 합니다.
- 오류 검출을 위해 default 절을 사용합니다.

The ending `else` is not mandatory. As for whether it is needed, it depends on what you want to achieve.

The trailing `else` clause will execute when none of the specified conditions is true. If the conditions are guaranteed to be [collectively exhaustive](#), then an `else` clause is entirely superfluous, except possibly to contain an assertion that catches the "impossible" condition. In your case, whether you need an `else` clause depends on whether you want specific code to run if and only if neither of `condition1`, `condition2`, and `condition3` are true.

- (if - else if - else)구문에서 else는 절대로 필수가 아닙니다.

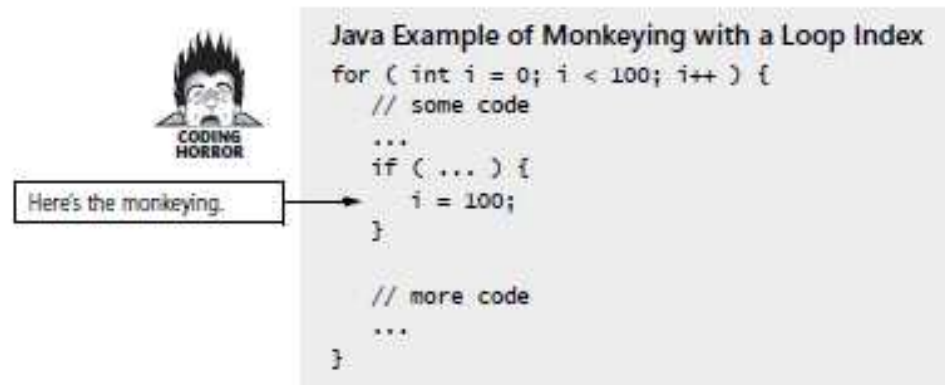
2. 반복문

1) For vs While

- 둘은 반복문의 제어여부(break , continue)로 사용 시기를 구분할 수 있습니다.

① For

- 간단한 상황에 사용하는 반복문
- 코드 내부에서 반복문의 제어를 하지 않고 , index가 완료 될 때 자동으로 끝납니다.
- 책에서는 for문을 break와 함께 사용하지 않는 것을 권장하지만 동의하지 않습니다.
- **절대로 index를 건드리는 코드를 사용하지 않습니다.** index를 건드려야 하는 경우 while문을 사용합니다.



② While

- 복잡한 상황에 사용하는 반복문
- 어떤 작업을 실행하다가 반복문을 빠져나가야 하는 상황에 사용합니다.

2) 반복문 가이드라인

- 반복문의 경우 다양한 실수로 인해 ERROR가 발생하기 쉽습니다.
- 아래의 여러 가지 가이드라인을 지켜 반복문의 ERROR를 미리 방지합니다.

① 반복문을 시작하기 전에 초기화 코드를 입력합니다.

② 무한 반복문은 While(true)가 표준 방식 입니다.

③ while문의 사용이 적절한 경우가 아니면 For문으로 반복문을 작성합니다.

- Index 부분에 반복 관련 코드가 모여 있으므로 변경 사항 수정이 간편합니다.

④ 반복문을 단순화 하고 또 단순화 합니다.

- 반복문에 영향을 주는 요소의 수를 최소화합니다.

3) Break 와 Continue

- 반복문 내에 수많은 break가 존재한다면 주의해야 합니다. 이런 경우 다수의 반복문으로 쪼개는 것을 권장하며 , break가 많다는 것은 오류와는 직접적인 관계가 없지만 예기치 못한 곳에서 탈출하여 개발자의 실수를 유발할 수 있는 시한폭탄 같은 존재입니다.
- C++의 label break나 C#의 goto는 사용할 수 있으면 사용합니다.

<읽기 좋은 코드가 좋은 코드다>

1. 주석

- 좋은 코드 > 나쁜 코드 + 해당 코드를 설명하는 주석
- 함수 명을 자세하게 쓰면 그 자체가 주석처럼 함수를 설명하기 때문에 주석을 달지 않아도 됩니다. 다시 말해, 좋은 함수 이름을 작성하고, 해당 이름으로 함수의 기능을 유추할 수 있다면 주석이 필요 없게 됩니다.
- 짧은 기능이어도 Update()에 코드로 작성하는 것보다 해당 기능을 함수에 넣고 Update에 넣으면 가독성이 상승하고 쉽게 이해합니다.

2. 주석에 적어야 할 사항

- TODO를 활용해서 코드에 현재 존재하는 결함을 적고, 해당 코드를 어떻게 수정할지에 대한 아이디어를 적습니다.
- 상수에 대한 설명이 필요하면 주석을 적습니다.
- 다른 사람이 제 코드를 읽었을 때, 이해하기 어려울 것 같은 내용에 주석을 적어줍니다.